

CSCI 432 Handout 10: Topological Sort

Name: CLASS

March 12, 2026

Topological Sort: The Algorithm

Topological sort takes a poset $(P, \leq)$ and returns a total order of all objects that is compatible with the partial order of the poset. By compatible, we mean that for all $a \leq b \in P$, we have a appears before b in the total order. For example, consider P to be a set of points in $\mathbb{R}^2$ and let the partial order be the product order; that is, $(a_x, a_y) \leq (b_x, b_y)$ if and only if $a_x \leq b_x$ and $a_y \leq b_y$. Given a set:

$$P = \{(0, 0), (\pi, \pi), (2, 4), (1, 2)\}$$

we can return the following total order:

$$P = [(0, 0), (1, 2), (2, 4), (\pi, \pi)].$$

Note that this is not the only compatible total order.

1. What other total order could have been returned?

Answer

$$(0, 0), (1, 2), (\pi, \pi), (2, 4)$$

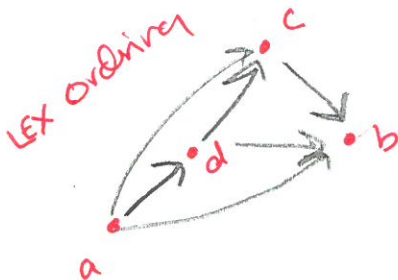
* If you find a total order compat. w/ Hasse diag, it is also compatible w/ orig. poset.

2. Another order we can impose on $\mathbb{R}^2$ is the lexicographical ordering (sometimes called the dictionary ordering), where $(a_x, a_y) \leq (b_x, b_y)$ if and only if $a_x < b_x$ or $(a_x = b_x \text{ and } a_y \leq b_y)$. What total orders on P exist that are compatible with the lexicographical ordering?

Answer

$$(0, 0), (1, 2), (2, 4), (\pi, \pi)$$

total order
The only order that is compatible
lex ordering
IFF
Hasse is a line



The following algorithm takes as input a poset, represented as a directed acyclic graph, and returns a compatible total order. If the poset orders $a \leq b$, then the DAG has an edge from vertex a to vertex b . For this algorithm, vertices have at least one attribute: outgoing, which stores a list of vertices that define the outgoing edges.

how we go
btwn poset + DAG
(adjacency list data structure)

Algorithm 1 Topological Sort

Input: $G = (V, E)$, a DAG

Output: L , a list of vertices in G in a total order compatible with the partial order of the DAG.

```

1: for  $v \in V$  do
2:   Add an attribute ' $v.count$ ' and initialize it to the indegree of  $v$ .
3: end for
4:  $Q \leftarrow$  queue containing the vertices with ' $count=0$ '.
5:  $L \leftarrow$  array of length  $|V|$ 
6:  $i \leftarrow 1$ 
7: while  $Q$  is not empty do
8:    $v \leftarrow Q.POP$ 
9:    $L[i] \leftarrow v$ 
10:   $i++$ 
11:  for  $u \in v.outgoing$  do
12:     $u.count--$ 
13:    if  $u.count = 0$  then
14:      Add  $u$  to  $Q$ 
15:    end if
16:  end for
17: end while
18: return  $L$ 

```

} initialize

pick a vertex to process

deal w/ v's neighbors

3. In Line 4, why is Q not empty? Let's prove this.

Answer Claim: Q is not empty on line 4.

Proof: By contradiction, assume that Q is empty.

Then, we know that all vertices have $\text{count} > 0$.
B/c lines 1-3 init count to be indegree, this means
that every vertex has an in-edge. Pick a random
vertex. It has an incoming edge. Walk backwards

4. Explain how topological sort is helpful for dynamic programming.

Answer

Use to determine
what order to
solve subproblems in.

to that vertex. It has
an incoming edge. Cont.
unit:

(1) find a vertex w/
no incoming. ✗

(2) find a vertex
we've seen before
 $\Rightarrow$ found a cycle ✗

(3) continue forever
(but, it was finite)

□

5. Walk through a small example.

Answer

For loop lns 1-3:

$$D_1: S \rightarrow \mathbb{N}$$

$$S \mapsto |V| - \# \text{verts w/ v.count defined}$$

Topological Sort: The Decrementing Function

Recall from HO-07 that a decrementing function is a function $D: S \rightarrow \mathbb{N}$, where S is the state space that such that (when there is no recursion) each time a loop is re-entered, the function is strictly less than the last time it entered that loop.

6. What is ^{the} the decrementing function for the for loop? (in Line 11)

Answer

$$D_2: S \rightarrow \mathbb{N}$$

$$S \mapsto |V| - \# \text{times the for-loop has completed iterated}$$



$$D_2: S \rightarrow \mathbb{N}$$

$$S \mapsto \sum u.\text{count} \text{ uEV. outgoing}$$

7. What is ^{the} the decrementing function for the while loop?

Answer

(here, don't worry if it decreases when we go through for loop of Lines 11-16.)

Topological Sort: The Loop Invariant

Next, we investigate partial correctness of the while loop. For this loop, what are the following statements (start with your best guess, then come back and revise if needed):

The loop guard G :

Q is not empty

$\neg G = Q$ is empty

The post-condition Q :

L is a total order of V ,
compatible with E

The pre-condition P :

Q is a queue of vertices w/ count=0

and Q is nonempty

and L is an array of length $|V|$, but not yet initialized
 $L[i]$ undec so far
v.i.

and $G=(V,E)$ is a DAG

and $i=1$

and $\forall v \in V, v.count = \# \text{ incoming edges (by LI for Lns 1-3)}$

The loop invariant $L = L_i$:

$X := \{w \in V \mid w \notin Q, w.count = 0\}$

$L[1 \dots i-1]$ contains a total order of X that
is compatible with $G \cap X$

$\hookrightarrow$ subgraph of G containing X

and

Can you use this loop invariant to prove partial correctness.

8. Initialization: If the preconditions are met and if we enter the loop, then the loop invariant is correct. $(P \wedge G \implies L)$.

Answer

9. Maintenance: If we entered the loop and the loop invariant held, then when we exit the loop, the loop invariant will still hold: $(G \wedge L_i \implies L_{i+1})$.

Answer

10. End: If the loop ends and the loop invariant was correct, then the algorithm is correct. $(L \wedge \neg G \implies Q)$.

Answer